

This time, when Python calls the `__init__()` method to create a new instance, it stores the make, model, and year values as attributes, like it did in the previous example. Then Python creates a new attribute called `odometer_reading` and sets its initial value to 0 ❶. We also have a new method called `read_odometer()` ❷ that makes it easy to read a car's mileage.

Our car starts with a mileage of 0:

---

```
2024 Audi A4
This car has 0 miles on it.
```

---

Not many cars are sold with exactly 0 miles on the odometer, so we need a way to change the value of this attribute.

## Modifying Attribute Values

You can change an attribute's value in three ways: you can change the value directly through an instance, set the value through a method, or increment the value (add a certain amount to it) through a method. Let's look at each of these approaches.

### Modifying an Attribute's Value Directly

The simplest way to modify the value of an attribute is to access the attribute directly through an instance. Here we set the odometer reading to 23 directly:

---

```
class Car:
    --snip--

my_new_car = Car('audi', 'a4', 2024)
print(my_new_car.get_descriptive_name())

my_new_car.odometer_reading = 23
my_new_car.read_odometer()
```

---

We use dot notation to access the car's `odometer_reading` attribute, and set its value directly. This line tells Python to take the instance `my_new_car`, find the attribute `odometer_reading` associated with it, and set the value of that attribute to 23:

---

```
2024 Audi A4
This car has 23 miles on it.
```

---

Sometimes you'll want to access attributes directly like this, but other times you'll want to write a method that updates the value for you.

### Modifying an Attribute's Value Through a Method

It can be helpful to have methods that update certain attributes for you. Instead of accessing the attribute directly, you pass the new value to a method that handles the updating internally.